# LakeCat

covers lakecat (0.1.0)

An Engine-Close Catalog Foundation for QueryGraph

Alexy Khrabrov

chiefscientist.org

*&*

*Codex with ChatGPT 5.5*

# Contents



# 1 LakeCat

## 1.1 Preface

LakeCat is a Rust-native Iceberg catalog foundation for QueryGraph. It starts from a deliberately conservative claim: the ordinary Iceberg REST catalog boundary must keep working for ordinary engines, but the next catalog also needs to become a governed control plane for Rust-first planning, semantic graph handoff, lineage, and agent access.

In this repository the catalog is LakeCat and the engine-side lakehouse implementation is Sail. The idea is not to invent a new table format or to make every client learn a new protocol. It is

to keep Iceberg compatibility at the boundary while moving the parts that need deep Iceberg knowledge closer to the Rust engine that can reason about them.

This book starts from first principles. It explains what a catalog is, why Apache Iceberg puts so much responsibility into metadata, what Sail already does with Iceberg metadata, and why a passive catalog is not enough for governed agentic systems. Then it shows the LakeCat shape: a thin Rust catalog that owns identity, tenancy, metadata-pointer state, policy gates, idempotent commits, and integration events, while delegating reusable engine, graph, and security work to Sail, Grust, and TypeSec.

The intended end state is QueryGraph.ai. LakeCat should become the catalog foundation for the next QueryGraph: a place where standard engines still see an Iceberg REST catalog, while QueryGraph can bootstrap Croissant, CDIF, OSI, ODRL, OpenLineage, TypeSec security receipts, and a Grust-backed graph from the same governed source of truth.

## 1.2 What A Catalog Is

A data catalog is often described as a place that lists datasets. That is true, but too small. A real catalog is the control plane between names, storage, metadata, identity, and intent.

At minimum, a catalog answers four questions:

1. What table does this name mean?
2. Where is its current metadata?
3. Who is allowed to read, write, plan, or administer it?
4. What changed, when, and under whose authority?

In a traditional database, the catalog is embedded inside the database system. The engine owns the table definitions, statistics, indexes, permissions, and transaction log. A client asks the database a question and the same system resolves names, checks permissions, plans the query, and executes it.

A lakehouse splits that system apart. Data files live in object storage. Metadata files live beside them. Multiple engines may read and write the same tables. A catalog becomes the agreement point: it maps a logical table name to the current metadata pointer and arbitrates updates to that pointer.

That pointer is deceptively important. If the catalog points at metadata version 17, the table is version 17. If a writer prepares version 18 and wins the compare-and-swap update, the table becomes version 18. If it loses, the table does not partially change. The catalog is not the table format, but it is the place where table history becomes visible and durable.

For human-scale analytics this can sound like bookkeeping. For agentic systems it becomes a trust boundary. A catalog can know the principal, the warehouse, the namespace, the table, the current snapshot, the requested columns, the row restriction, the storage profile, and the policy receipt. If that information is captured before planning and committed after state changes, the catalog becomes the control plane for governed data access rather than a passive address book.

## 1.3 What Iceberg Does

Apache Iceberg is a table format for large analytic tables. Its core design is simple: put the table's truth in explicit metadata files, and let engines use that metadata to plan reads and validate writes without relying on directory listing or fragile storage conventions.

An Iceberg table has a current metadata file. That metadata names schemas, partition specs, sort orders, snapshots, properties, and the current snapshot. Snapshots point to manifest lists. Manifest lists point to manifests. Manifests describe data files and delete files. The data files normally use formats such as Parquet, Avro, or ORC.

This layered metadata gives Iceberg its practical power:

- Schema evolution is explicit.
- Partition evolution is explicit.
- Snapshot isolation is explicit.
- Commit conflicts can be checked before the current pointer advances.
- Scan planning can prune manifests and files before touching data.
- Deletes can be represented without rewriting every data file immediately.
- Multiple engines can interoperate because the table state is stored in a shared, documented format.

The catalog role in Iceberg is intentionally narrow. Standard clients need to load table metadata, create namespaces and tables, commit changes, and sometimes receive credentials or scan tasks. The catalog must not require business semantics or proprietary metadata for normal table access. If standard clients have to call a non-standard endpoint to read an ordinary table, compatibility is already broken.

LakeCat preserves that rule. Iceberg metadata stays pristine. Business semantics, policy, graph, lineage, and agent state are derived control-plane or graph data. The table remains an Iceberg table.

## 1.4 The Current Catalog State

The current lakehouse catalog market is converging on a simple fact: the catalog is no longer a sidecar. It is the place where table identity, tenancy, commits, credentials, governance, and interoperability are negotiated.

Hive Metastore gave early lakehouses a familiar namespace and table registry, but it was born for a different format era. Cloud warehouses built proprietary catalogs around their own engines. Nessie explored Git-like table references and branching. Unity Catalog turned governance and sharing into a central product surface. Lakekeeper has shown how a modern open Iceberg REST catalog can model warehouses, projects, credentials, soft deletion, and management APIs without polluting table metadata. Polaris has made the strongest standards-centered move: an open Iceberg REST catalog with vendor gravity, clear governance ambition, and a credible route for many engines to meet at the same catalog boundary.

That is why Polaris is a winner. It is not because it is the last word in catalog architecture. It is winning because it occupies the obvious shared surface at the right moment:

- It speaks Iceberg REST instead of asking every engine to learn a new table protocol.
- It treats the catalog as a security and governance surface, not merely a pointer map.
- It gives enterprises an open center of gravity around Iceberg while the warehouse, query-engine, and cloud-storage layers remain plural.
- It can be adopted incrementally: engines can keep reading tables, while operators gain a real control plane.

LakeCat should learn from that. The winning move is not to reject Polaris-style compatibility. The winning move is to keep that compatibility and then ask what a Rust-native, QueryGraph-facing catalog can do when it is allowed to plan near Sail, emit graph through Grust, and ask TypeSec for authorization proofs.

## 1.5 What Intermediation Loses

Catalogs win by sitting between engines and data. That same position can also flatten the system.

When the catalog is only an intermediary, it often sees the table name, the current metadata pointer, the caller, and perhaps a credential request. The engine sees the schema, manifests, statistics, deletes, partition evolution, scan filters, and physical plan. The governance system sees policy. The lineage system sees an event after the fact. The semantic layer sees a separate model. Each system receives a shard of the truth.

The loss is not merely elegance. It is operational:

- Policy can be checked before access but not carried into scan planning.
- Credentials can be vended without proving why a raw credential exception was allowed.
- Lineage can describe that something happened but not bind to the exact governed plan, snapshot, policy, and table metadata.
- Semantic layers can drift from the physical tables they describe.
- Agents can receive broad storage access when they should have received a governed plan and short-lived, narrow task set.
- Engines can optimize with file statistics while catalogs remain blind to the consequences of those choices.

The catalog should not become a replacement engine. But it should not be a blind intermediary either. LakeCat's thesis is that the catalog can stay thin and standard while still becoming engine-close at the moments where correctness, security, and semantics depend on planning evidence.

## 1.6 Why Sail Matters

Sail is the Rust lakehouse engine path. It already contains the pieces that should understand Iceberg deeply: catalog abstractions, generated Iceberg REST models, DataFusion table providers, manifest pruning, metadata-as-data paths, write and commit plumbing, and format-version checks.

That matters because scan planning and commit validation are not generic catalog chores. They require knowledge of Iceberg schemas, projections, partition specs, sort orders, snapshots, manifests, data files, delete files, statistics, and expression models. If LakeCat reimplements those

details, it becomes a second Iceberg engine. The same bug class appears twice, and the catalog begins to drift away from the planner that actually executes work.

LakeCat takes the opposite path. Put reusable Iceberg and planning behavior in Sail. Let LakeCat call Sail for the parts that require engine-grade knowledge. Keep LakeCat responsible for the catalog boundary: identity, tenancy, durable state, policy gates, idempotency, audit, outbox, and standard REST compatibility.

The current LakeCat architecture follows this line. The service exposes an Iceberg REST-compatible surface under `/catalog/v1`. The Sail-facing engine path handles scan planning, table-status conversion, metadata preparation, manifest expansion, and standard response validation. LakeCat keeps only the additional context it must own: which principal asked, which policy narrowed the read, which metadata pointer was current, which commit won, and which event should be replayed to graph and lineage sinks.

## 1.7 Why Move Catalog Work Closer To The Engine

A passive catalog returns metadata locations and lets the client plan. That is compatible, but it is not enough for the next generation of governed systems. Agents, notebooks, services, and model pipelines need stronger guarantees:

- A policy should be enforced before a scan is planned.
- Column restrictions should narrow the projection before file tasks are produced.
- Row predicates derived from policy should become mandatory scan filters.
- A stateless `fetchScanTasks` call should not widen a prior governed plan.
- Credentials should be short-lived, scoped, and audited.
- Commit retries should be idempotent and should not replay a different request.
- Graph and lineage side effects should reflect committed catalog state, not best-effort handler side effects.

Those guarantees sit between catalog state and engine planning. If the catalog is too far from the engine, it can check a policy and then hand the client a metadata pointer, hoping the client preserves the restriction. If the engine is too far from the catalog, it can plan efficiently but may not know the governance receipt or durable identity context.

LakeCat fuses those two moments. LakeCat authorizes the request, derives the effective restriction, and asks Sail to plan or validate against the current metadata pointer. Sail performs the Iceberg work. LakeCat returns the standard shape and records the governance evidence.

The result is still an Iceberg catalog. The difference is that governed reads and writes are planned through the same Rust implementation that QueryGraph will use downstream.

## 1.8 LakeCat's Thin Boundary

LakeCat should be thin, but thin does not mean trivial. It owns the durable catalog state that must be correct even when external sinks are unavailable.

The core LakeCat responsibilities are:

- Serve the Iceberg REST Catalog API for standard clients.

- Model projects, warehouses, namespaces, tables, views, and storage profiles.
- Persist metadata pointers and compare-and-swap commit history.
- Validate idempotency keys and replay only matching commit bodies.
- Resolve request identity from headers, bearer tokens, agents, and TypeDID envelopes.
- Ask TypeSec for authorization decisions and persist receipts.
- Route scan planning and commit preparation through Sail.
- Record audit and outbox events inside the catalog transaction.
- Drain committed events to Grust and OpenLineage sinks.
- Publish a QueryGraph bootstrap bundle.

The deliberately excluded responsibilities are just as important:

- LakeCat does not invent a table format.
- LakeCat does not fork Iceberg manifest pruning.
- LakeCat does not own graph traversal or graph query behavior.
- LakeCat does not own security semantics or agent trust semantics.
- LakeCat does not author QueryGraph's final business semantic model.

That boundary gives each sibling project a clear job. Sail owns reusable Iceberg and planning behavior. Grust owns graph schema, storage, and query mechanics. TypeSec owns policy, capabilities, TypeDID envelopes, secure agents, and authorization semantics. QueryGraph owns the semantic application built on top.

## 1.9 The Read Path

The LakeCat read path begins like a standard catalog request and ends with a governed Sail plan.

1. A client asks to load or plan a table through the Iceberg REST surface.
2. LakeCat resolves the warehouse, namespace, table, and current metadata pointer.
3. LakeCat resolves the request identity.
4. LakeCat asks TypeSec whether the principal can perform the requested action.
5. TypeSec returns a decision and any enforced restrictions.
6. LakeCat turns those restrictions into a `ReadRestriction`: allowed columns, required row predicate, purpose, policy hash, and audit context.
7. LakeCat asks Sail to plan against the current metadata pointer with the effective projection and filters.
8. Sail validates Iceberg expressions against generated REST models and table schema, expands manifests, applies conservative file-bound pruning when metrics are present, and carries delete-file references into file scan tasks.
9. LakeCat returns Iceberg-compatible plan and task responses.
10. LakeCat records audit and outbox events that can later be projected into graph and lineage.

The important detail is that the policy restriction becomes part of planning, not a note beside it. An empty client projection under a column restriction means the allowed columns. A client projection can narrow further, but cannot widen. LakeCat records both the client's requested projection and the effective projection that survived the server-derived column restriction in the durable scan-planned replay evidence. The same rule applies to stats-field requests: LakeCat records both

the client's requested stats fields and the effective stats fields that survived the restriction, while the compatibility `stats-fields` extension remains the narrowed effective set. The default REST path is tested at the Sail boundary: Sail receives only the effective projection and mandatory policy filters, while LakeCat keeps the broader request and narrowed result as replay evidence. During `fetchScanTasks`, LakeCat recomputes the current restriction and sends Sail the required projection and mandatory filters again; the response extension and audit outbox record the same proof. A stale or legacy token cannot silently expand back to all columns.

## 1.10 The Commit Path

The write path follows the same principle: LakeCat owns the catalog transaction, Sail owns reusable Iceberg validation and metadata preparation.

1. A client sends an Iceberg commit request, optionally with an idempotency key.
2. LakeCat validates the request shape and the idempotency key.
3. LakeCat resolves identity and asks TypeSec for the commit capability.
4. If the idempotency key already has an exact stored response, LakeCat returns that response before Sail validation or metadata-object writes.
5. LakeCat loads the current metadata pointer from the store.
6. LakeCat delegates Iceberg update validation and metadata assembly to Sail.
7. LakeCat rejects metadata-object writes that target the table's current metadata pointer, so the current metadata file cannot be overwritten before the store commit has won.
8. LakeCat rejects metadata-write plans that do not carry a concrete new metadata location.
9. LakeCat rejects metadata-object locations outside the table's matched storage profile prefix, and also rejects the storage-profile root itself because metadata commits must create a child object.
10. LakeCat rejects literal or percent-encoded dot path segments in metadata object locations, so commit plans cannot rely on traversal-like spelling to address anything other than a plain child object.
11. LakeCat writes the new metadata object through the warehouse storage profile with create-only object-store semantics.
12. LakeCat advances the table pointer with compare-and-swap.
13. LakeCat persists idempotency, audit, pointer-log, and outbox records.
14. If the store rejects the commit after a local metadata write, LakeCat cleans up the uncommitted metadata object when it can do so safely.
15. Outbox draining projects the committed event to graph and lineage sinks.

The cleanup path is deliberately secondary to the commit result. If metadata cleanup fails after the store rejects a commit, LakeCat preserves the original store or compare-and-swap error class and appends cleanup context. A stale pointer conflict still looks like a conflict to an Iceberg client, but the message carries SHA-256 hashes of the expected and actual metadata locations so operators can diagnose the race without exposing raw object paths. The service-level regression for this path checks the API response and the filesystem side effect together: the rejected metadata object is gone, while the conflict body contains only hashed pointer evidence. True cleanup failures use the same redaction discipline: the cleanup context identifies the uncommitted metadata object by

`metadata-location-hash=sha256:...`, not by the raw object path. When that cleanup failure is appended to the preserved commit conflict, LakeCat keeps only `error-detail-hash=sha256:...` evidence for the cleanup detail, so raw backend text cannot leak through the combined conflict message. If cleanup discovers the uncommitted object is already absent, LakeCat treats that as successful cleanup rather than turning a resolved orphan into an internal error. Cleanup also refuses to delete the previous committed metadata pointer if a future plan accidentally reports it as the staged write; the committed metadata object remains the table's current state, not an orphan. The same audit-safe shape applies before the write: current-pointer overwrite, existing-object overwrite, unsupported object-store configuration, and storage-profile-prefix failures report metadata-location hashes, and prefix mismatches also report a storage-profile-prefix hash rather than raw object paths. A root-targeted metadata write uses the same redacted error shape: the operator sees that the plan did not name a child metadata object without receiving the raw table or storage root. Dot-segment failures use the same style: literal `..` and percent-encoded `%2e%2e` paths fail before object-store writes and expose only the metadata-location hash. Decorated metadata object locations with URI query strings, fragments, or URI userinfo are rejected at the same pre-write boundary, so a commit plan cannot smuggle version selectors, backend hints, fragment markers, or embedded credentials into what should be a plain metadata object address.

Idempotency is part of correctness. Reusing the same key for the same commit can return the stored response even after the table has advanced beyond the original commit requirements. Reusing the same key for a different body must conflict. LakeCat persists a normalized request hash and stores only audit-safe evidence, not raw secrets or raw idempotency keys. REST idempotency keys are intentionally narrow: `x-lakecat-idempotency-key` must be 1 to 128 ASCII characters and may use only letters, digits, `-`, `_`, `.`, or `:`. Invalid keys, including non-ASCII or invalid header bytes, fail before LakeCat performs authorization, Sail validation, table loading, or metadata-object writes. When a reused key is attached to a different commit body, the conflict response also stays redacted: it does not echo the raw key or the mismatched metadata object location. The Turso spine pins the same redaction for both commit-time reused-key conflicts and explicit idempotency replay probes: the raw key, mismatched request hash, and mismatched metadata object location are not operator-facing error text. The service regression for this path proves the replay happens before metadata-object writes: an exact retry returns the stored response without touching the already committed metadata object. The same regression pins the outbox side effect: exact replay and mismatched reused-key conflicts leave only the original `table.commit` outbox event, so QueryGraph and OpenLineage replay do not see duplicate commit work from retry traffic. Another regression sends a commit whose requested metadata location is the table's current pointer and verifies that LakeCat returns a bad request without touching the existing metadata file. Another sends a commit to a different metadata location that already exists and verifies that LakeCat returns a conflict without overwriting that non-current object. The same guard fails closed if a future Sail plan asks LakeCat to write metadata but does not provide a new object location, or if it tries to use the storage profile root as the new metadata object. A companion regression rejects both literal and percent-encoded dot path segments in a planned metadata location. When a backend object store fails setup, create-only write, or cleanup, LakeCat keeps the metadata location hash and adds `error-detail-hash` evidence instead of returning

raw backend text. That includes invalid metadata URI parsing and unsupported backend setup failures: the response names the hashed metadata location and hashed backend detail, not the submitted path, object name, scheme, or parser/backend diagnostic. That matters for local files, cloud bucket keys, and credential-provider diagnostics: operators can correlate a failure without copying sensitive storage topology into API responses or logs.

The embedded in-memory store follows the same commit evidence contract as the Turso path. A successful commit emits one `table.commit` audit/outbox event with the compact commit record, authorization receipt, response hash, and redacted idempotency-key hash. An idempotent replay returns the stored response without adding a second outbox event, so tests and local embedded deployments exercise the same outbox invariant as the durable spine.

Commit records also carry a response hash over the stored table response. That pair matters: the request hash proves which commit body won or replayed, while the response hash proves which metadata pointer and table body LakeCat returned to clients and later projected through graph and lineage replay.

The same commit record includes compact summary evidence: Iceberg format version, current snapshot id, and the policy hash from the authorization receipt when one exists. QueryGraph can inspect those fields from the pointer-log/outbox stream without parsing full table metadata for every catalog audit question.

Operators and QueryGraph can read that pointer-log evidence through a governed management endpoint:

```
curl -s \
  -H 'x-lakecat-principal: operator@example.com' \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/tables/
events/commits
```

The response contains compact commit records: sequence number, previous and new metadata locations, request hash, response hash, idempotency-key hash, Iceberg format version, current snapshot id, policy hash, principal, and a commit hash. The read itself enters the durable outbox as `table.commits-listed` and drains as lineage evidence plus catalog-facing `Commit` graph anchors keyed by table and sequence number. That gives audit tools and QueryGraph a Grust-visible pointer-log inspection trail without requiring direct access to the Turso catalog database or making LakeCat a graph query engine. QGLake acceptance now exercises this path directly: the fixture issues an idempotent no-op commit probe, reads the compact commit-history endpoint, verifies that the record preserves the table's Iceberg format-version and current snapshot summary, and then requires the lineage drain to replay `table.commits-listed` receipt hashes plus compact commit count, sequence-number, and commit-hash summary fields before the QueryGraph handoff is accepted.

## 1.11 The Durable Spine

LakeCat's durable local spine uses the Rust `turso` crate behind the `turso-local` feature. The store contract remains portable, but the local foundation is not SQLx. The important tables are not an application afterthought; they define the catalog's control-plane memory:

- projects and warehouses;
- storage profiles;
- namespaces and tables;
- metadata pointer log;
- idempotency records;
- soft deletes;
- policy bindings;
- audit events;
- outbox events.

Object storage remains the source of Iceberg metadata files. Turso stores the atomic pointer, management state, idempotency evidence, and event record. This mirrors the Iceberg catalog contract: metadata files describe the table; catalog state decides which metadata file is current.

Outbox draining is intentionally strict. LakeCat projects a batch to graph and lineage sinks first, then acknowledges the whole projected batch in the store. If projection fails, nothing is acknowledged. If the store reports that fewer events were acknowledged than LakeCat projected, the drain fails with an acknowledgement mismatch instead of returning a quiet partial success. That keeps retry and operator evidence honest when a concurrent drain or backend anomaly interferes with delivery accounting.

## 1.12 Grust For Graph Concepts

Catalog events naturally form a graph. A server contains projects. A project contains warehouses. A warehouse contains namespaces and storage profiles that define credential roots. A namespace contains tables. A table has columns, snapshots, manifests, files, policies, commits, scan plans, principals, and lineage runs. QueryGraph needs that graph, but LakeCat should not become a graph database.

Grust owns the graph layer. It is the place for reusable graph taxonomy, projection builders, graph stores, traversal indexes, Cypher support, and typed or untyped graph operations. LakeCat's responsibility is narrower: translate committed catalog events into a bounded envelope and pass it through a catalog-facing sink.

In practice this means LakeCat can emit graph events for stable catalog facts:

```
Server CONTAINS Project
Project CONTAINS Warehouse
Warehouse CONTAINS Namespace
Namespace CONTAINS Table
Table HAS_COLUMN Column
Table GOVERNED_BY Policy
Warehouse HAS_STORAGE_PROFILE StorageProfile
Principal CAN_PLAN ScanPlan
Commit DERIVED_FROM Snapshot
LineageRun USED_BY QueryGraphModel
```

High-cardinality file and manifest facts should stay queryable through Iceberg/Sail metadata-as-data unless Grust provides a reusable taxonomy and storage strategy for them. The graph should be powerful, but the catalog must not smuggle a second lakehouse engine into its event sink.

The current local direction already proves the boundary: LakeCat's `grust-local` sink calls Grust-owned LakeCat projection helpers, and the Grust Cypher boundary test verifies catalog graph projection without making LakeCat own Cypher parsing, traversal, or graph execution. The current boundary test writes table-adjacent `Column`, `Snapshot`, and `Commit` events plus `Principal`, `ScanPlan`, tenant-root `Server`, and credential-root `StorageProfile` catalog events through Grust, then matches catalog-event labels through Grust Cypher. Storage profile replay uses redacted evidence such as `secret-ref-present` and the secret-reference provider, never the full secret-store URI. Credential-vend attempts replay through that same thin boundary as `StorageProfile` graph events keyed by the redacted credential-root anchor, so QueryGraph can see a principal attempted credential-root access without seeing a secret reference or raw credential material. That proves QueryGraph can discover the semantic anchors LakeCat emits while the richer node/edge materialization remains reusable Grust work.

## 1.13 TypeSec For Security

LakeCat is a policy enforcement point, not the author of security semantics. TypeSec owns the semantics: RBAC, ODRL-style policy composition, typed capabilities, TypeDID envelopes, secure agents, credential issuance decisions, and authorization proofs.

Every externally meaningful action should pass through TypeSec:

- catalog configuration reads;
- namespace creation and listing;
- table creation, load, scan planning, commit, drop, and restore;
- credential vending;
- policy management;
- graph and lineage reads.

LakeCat gathers the request context and asks TypeSec for a decision. The context can include principal DID, agent DID, bearer-derived subject, warehouse, namespace, table, columns, snapshot, requested credential duration, purpose, and active policy bindings. TypeSec returns a decision and receipt. LakeCat persists the receipt with audit-safe hashes and applies the resulting restrictions before Sail plans.

This is where ODRL becomes operational. An ODRL-style policy can say that a principal may read only certain columns, only for a purpose, or only with an enforced row predicate. LakeCat parses the minimal enforceable subset it needs to narrow the plan, but policy composition and authorization semantics belong to TypeSec. LakeCat should ask TypeSec, not grow a parallel security language. When that subset is expressed through ODRL constraints, LakeCat accepts only operators that actually mean "use this as the allowed or narrowing value"; missing or deny-shaped operators fail closed instead of being treated as governed read permission. The bounded parser accepts camel-case, kebab-case, and prefixed JSON-LD operand keys such as `odrl:leftOperand` and `odrl:rightOperand`. It also accepts compact JSON-LD term objects such

as `{"@id":"odrl:eq"}` for constraint operands and operators, plus JSON-LD `@value` and `@list` right operands for bounded allowed-column, purpose, and credential-TTL values. Malformed JSON-LD lists still fail closed, and the parser does not turn LakeCat into a full ODRL reasoner. Recognized constraint operands must also include a right operand; otherwise LakeCat rejects the policy material instead of silently dropping an allowed-column, row-predicate, purpose, or credential-TTL restriction. The service route pins this behavior too: a table scan with a malformed active ODRL restriction, including malformed JSON-LD allowed-column lists, fails before Sail planning and before `table.scan-planned` replay evidence is emitted. A `fetchScanTasks` call with the same malformed JSON-LD active policy fails before Sail fetch execution and before `table.scan-tasks-fetched` replay evidence is emitted, and a credential request with the same malformed JSON-LD active policy fails before issuer dispatch and before `credentials.vend-attempted` replay evidence is emitted. Purpose is composed the same way: every purpose source in the active policy material must agree. If one binding says a read is for `resilience-demo` and another says `training`, LakeCat rejects the restriction instead of guessing which purpose should follow the agent into Sail planning, credential TTL proof, and QueryGraph handoff evidence.

Credential vending follows the same rule. Raw credential vending is an audited exception. Governed Sail-planned reads are the default path for agents and untrusted principals. When credentials must be issued, TypeSec checks the `credentials.issue` capability for the exact secret reference and LakeCat returns only scoped, short-lived credential configuration. If policy carries a `max-credential-ttl-seconds` restriction, LakeCat passes that cap to the credential issuer and annotates each returned credential with `lakecat.max-credential-ttl-seconds`, so the exception path has an auditable duration bound. If the cap appears in multiple supported ODRL locations in the same policy document, or across multiple active policy bindings, LakeCat keeps the tightest value before asking the credential issuer. If an issuer returns that LakeCat TTL key itself, LakeCat normalizes the response to one TTL entry per credential and keeps the stricter valid TTL, so duplicate backend-supplied entries cannot widen or confuse the policy cap. The same response boundary owns the rest of the LakeCat evidence: issuer-supplied values for `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, `lakecat.authorization-principal`, and `lakecat.governed-read-required` are removed and replaced with catalog-derived values before the response is returned. The REST credential-vending regressions exercise this at the public response boundary: a backend can return multiple TTL entries or forged catalog evidence, but `loadCredentials` exposes one canonical proof while preserving issuer-owned credential details such as credential kind and provider session tokens. LakeCat records the same decision shape in audit/outbox evidence without copying raw credentials: each vended credential gets a hashed prefix, canonical LakeCat evidence values, and a hash of issuer-owned config. Replay can prove the response posture, but it does not inherit cloud session tokens.

## 1.14 Rust-First Engines And The V3 To V4 Path

The new Rust-first engine path matters because it changes where catalog intelligence can live. Sail is not a Java service with Rust bindings bolted on the side. It is a Rust engine path built around

Arrow, DataFusion, generated Iceberg REST models, catalog provider traits, manifest pruning, metadata-as-data scans, and table-status conversion.

That shape gives LakeCat a better option than reimplementation. LakeCat can ask Sail for typed Iceberg behavior instead of parsing just enough JSON to survive a request. That matters for Iceberg v3 and the emerging v4 work. Format v3 already pushes catalogs and engines toward richer metadata, row lineage, deletion semantics, and better interoperability around advanced table state. Format v4 is still settling, but it is plainly moving the lakehouse toward more adaptive and structured metadata rather than less.

LakeCat should evolve under three rules:

1. Conform to Iceberg v3 for ordinary clients.
2. Preserve unknown and emerging v4 metadata without claiming settled semantics.
3. Prefer typed Sail support as soon as Sail exposes it, using JSON passthrough only as a compatibility bridge.

That gives LakeCat room to support v4-ready capability flags, round-trip tests, metadata extension preservation, and future metadata-tree planning without forking Iceberg. The catalog can become more intelligent while the table remains portable. The standard path stays boring. The governed Sail path becomes richer.

The current v4 bridge is intentionally narrow and tested as such. When LakeCat sees `format-version: 4`, it does not pretend that Sail already has a settled typed v4 model. Instead, `lakecat-sail` extracts the stable JSON envelope fields that remain useful across versions: table UUID, location, schema id, snapshot id, sequence number, manifest-list path, default spec, and field names. It can plan a governed manifest-list scan task from that envelope and validate the signed plan task again during `fetchScanTasks` so a stateless fetch cannot drift to a different manifest list or widen the governed projection and filters. It also validates stable commit requirements such as table UUID, current schema id, main snapshot id, last assigned field id, and default spec id. Pruning and typed metadata-tree semantics wait for Sail-owned v4 support.

## 1.15 OSI, OpenLineage, And Responsible Semantic Handoff

QueryGraph needs more than physical table access. It needs a semantic picture: datasets, fields, policies, lineage, graph relationships, and anchors that can survive import. LakeCat should publish that picture without pretending to be QueryGraph.

The LakeCat bootstrap bundle contains:

- Semantic Croissant and CDIF projections for dataset and field discovery.
- An OSI handoff with stable dataset and field anchors.
- ODRL policy artifacts and TypeSec policy context.
- OpenLineage events for catalog changes, scan plans, commits, and maintenance.
- A Grust-ready graph envelope.
- A manifest that hashes each emitted artifact.

The OSI boundary is deliberately careful. LakeCat should not author rich business metrics, dimensions, joins, ontology claims, or authoritative semantic names. It can publish stable anchors and governed source metadata. QueryGraph owns the final semantic model.

This is the Responsible Semantic Layer boundary. Semantic Croissant and CDIF make datasets and fields discoverable and exchangeable. OSI gives QueryGraph a stable handoff for semantic anchors without forcing LakeCat to own business meaning. OpenLineage records how catalog and planning events happened. Together they let the semantic layer be responsible because it can be traced back to catalog state, policy, and lineage, not just to a hand-authored model file.

OpenLineage fits the catalog outbox. A committed table create, scan plan, commit, soft delete, restore, or maintenance action can become a lineage event. Because the event is drained from a durable outbox after the catalog transaction, lineage reflects committed state rather than a handler's best-effort side effect. Drains process pending events in `created_at,event_id` order before projection, response summarization, and delivery acknowledgement. That stable order is part of the replay contract QueryGraph and OpenLineage consume, and it holds even when a custom store implementation returns a pending batch in another order.

## 1.16 QueryGraph.ai When LakeCat Is Done

QueryGraph.ai is the enterprise lakehouse this work is pointing toward. QueryGraph needs the catalog to be more than a storage address book. It wants to answer questions over data, metadata, semantics, policy, and lineage as one governed graph. That requires a catalog foundation that can speak to ordinary Iceberg clients and also publish trustworthy control-plane facts.

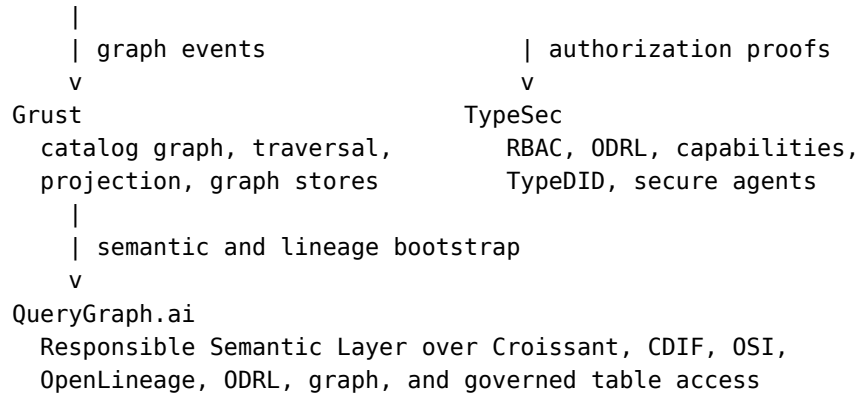
LakeCat supports that foundation by exposing a QueryGraph bootstrap endpoint:

```
/querygraph/v1/bootstrap
```

The bundle gives QueryGraph an import contract. QueryGraph can verify hashes, load catalog graph envelopes through Grust, inspect policy artifacts through TypeSec, and attach semantic modeling work to stable dataset and field anchors. The import path should prove that LakeCat is the substrate, not a standalone demo.

When LakeCat is done, the QueryGraph.ai architecture looks like this:

```
Standard engines and tools
  Spark, Trino, Flink, PyIceberg, notebooks
    |
    | Iceberg REST
    v
LakeCat catalog
  identity, tenancy, metadata pointers, commits, policy gates,
  idempotency, credential-vending decisions, audit, durable outbox
    |
    | typed planning and table semantics
    v
Sail
  Rust-native Iceberg planning, metadata-as-data, scan pruning,
  delete handling, commit validation, table maintenance
```




Basic catalog use remains optional and standard. A normal Iceberg engine can load and commit tables without knowing QueryGraph exists. The enhanced path is there for enterprises that want more: governed Sail-planned reads, TypeSec authorization receipts, ODRL rights, TypeDID agent identity, OpenLineage replay, Semantic Croissant/CDIF publication, OSI handoff, and Grust graph loading.

That is the core motivation for LakeCat. The next QueryGraph.ai should not bolt governance, graph, and lineage onto tables after the fact. It should begin from a catalog that already records governed state transitions and exposes standard, engine-close planning.

## 1.17 Implementation Shape

The current workspace shape expresses the architecture directly:

```
crates/
  lakecat-core        stable IDs, errors, time, config, content hashes
  lakecat-api         Iceberg REST request/response adapters
  lakecat-store       catalog state traits and Turso-backed implementation
  lakecat-sail        Sail provider bridge and privileged planning client
  lakecat-graph       catalog-facing Grust sink/adapters
  lakecat-security    TypeSec integration and authorization receipts
  lakecat-lineage     OpenLineage projection and event receipts
  lakecat-querygraph  Croissant/CDIF/OSI/ODRL/OpenLineage bootstrap projection
  lakecat-service     axum service, middleware, auth, routing
  lakecat-cli         admin, local demo, conformance, bootstrap export
```

Feature gates keep integrations honest:

```
sail-local    use local Sail APIs for planning and provider integration
typesec-local use local TypeSec APIs for governance and TypeDID verification
grust-local   use local Grust APIs for catalog graph projection
turso-local   use the Turso-backed durable store
```

Embedded defaults stay safe for tests. Real integrations are explicit. That matters because LakeCat is a foundation, not a pile of optional demos. A test that only uses memory stores should not accidentally depend on a sibling repo. A test that claims to validate TypeSec should enable `typesec-local` and call TypeSec.

The dependency contract is executable because LakeCat still has one active sibling bridge. Grust and TypeSec now resolve from the published `grust-graph` 0.9.0, `grust-cypher` 0.9.0, and `typesec` 0.8.0 crates, so the `grust-local` and `typesec-local` features no longer require sibling checkouts merely to compile. That makes the graph and governance boundaries reproducible outside this machine while still keeping their reusable behavior in Grust and TypeSec.

Sail is different today: LakeCat still uses local Sail paths plus a checked-in patch bridge for helper APIs that are not yet published. Before pushing a slice that touches integration features, run:

```
scripts/check-local-dependency-contract.sh
```

The script checks the manual-only CI trigger, scans every GitHub workflow file for forbidden automatic cloud triggers, verifies crates.io resolution for the published Grust graph/Cypher and TypeSec versions, the local Sail path bridge, the Sail patch files manual CI applies, and the concrete Sail helper API surface LakeCat uses: generated Iceberg REST models, typed metadata inputs, planning result helpers, fetchScanTasks result helpers, and table-status conversion. It also checks the local QueryGraph Rust importer for the LakeCat view receipt-chain contract: `receipt-chain-hash` must be preserved in view receipt evidence and missing receipt-chain evidence must fail closed. Manual-only means no automatic push, pull-request, pull-request-target, merge-queue, repository-dispatch, scheduled, workflow-run, or reusable-workflow cloud runs; the local audit fails if any of those triggers appear before the local gates are proven stable. It is not a substitute for upstreaming the Sail helper APIs or re-enabling automatic CI; it is a guard that makes drift visible while LakeCat still depends on unpublished Sail helper work and a local QueryGraph acceptance target.

As of the current local reconciliation, the Sail helper work is not an anonymous dirty tree. `/Users/alexy/src/sail` has scoped local commits on `codex/graph` for exposing Iceberg REST models to LakeCat, preserving Iceberg manifest lower and upper bounds in Avro, and adding Sail's Cypher graph query extension. That Cypher extension is a Sail SQL/analyzer/planning surface; the catalog graph taxonomy, projection helpers, traversal, and stores remain Grust responsibilities. The only remaining Sail working-tree entries are untracked artifact/book directories, and pushing the Sail branch upstream is blocked by HTTPS GitHub authentication rather than by local test failures.

## 1.18 Standard Compatibility And Extensions

LakeCat must be boring where standards require boring behavior. Standard Iceberg clients should be able to use the catalog without knowing QueryGraph exists. Business semantics and agent state must not be required custom Iceberg metadata.

Extensions belong beside the standard path:

- `/catalog/v1` serves Iceberg REST compatibility.
- management APIs handle warehouses, policies, profiles, and operational state.
- `/querygraph/v1/bootstrap` publishes QueryGraph import artifacts.
- feature-gated Sail paths provide governed scan planning and local provider integration.

Format v4 work should follow the same rule. Prefer typed Sail support when available. JSON passthrough can bridge compatibility, but it is not the desired long-term implementation. Round-trip tests should prove LakeCat preserves unknown or evolving metadata without claiming settled semantics too early.

## 1.19 Workflow Examples

The catalog is easiest to understand by watching it participate in ordinary work. LakeCat should not ask users to think about graph, lineage, security, and Sail every time they read a table. Those systems should appear when they matter: at the boundary where a name is resolved, a policy is enforced, a plan is created, credentials are withheld or issued, and a durable event is replayed.

The examples below use one table, `local.default.events`, but the pattern is the same for larger warehouses. The important point is not the exact sample data. It is the catalog role in each workflow.

### 1.19.1 Starting The Catalog

A local operator starts LakeCat as an Iceberg REST catalog plus management surface:

```
cargo run -p lakecat-service --features sail-local,turso-local,typesec-
local,grust-local
```

The standard catalog path is still `/catalog/v1`. The management and QueryGraph surfaces sit beside it:

```
/catalog/v1
/management/v1
/querygraph/v1/bootstrap
```

A simple health-oriented configuration read shows the split. Standard engines care about the Iceberg endpoints. Operators and QueryGraph care about the management and bootstrap endpoints.

```
curl -s http://127.0.0.1:3000/catalog/v1/config
```

At this point the catalog is already doing more than route HTTP. It has a warehouse identity, a store, a governance engine, a Sail planning seam, a graph sink, and a lineage sink. Embedded defaults keep the local loop small, but the same trait boundaries can point to Turso, TypeSec, Grust, and Sail.

### 1.19.2 Registering The Warehouse Shape

An operator usually starts with management objects. A server groups projects. A project groups warehouses. A warehouse owns namespaces, tables, views, storage profiles, policy bindings, and the metadata pointer state that standard engines see through Iceberg REST.

```
curl -s -X PUT http://127.0.0.1:3000/management/v1/servers/prod \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Production LakeCat",
    "endpoint-url": "https://lakecat.example.com",
    "properties": {
```

```
      "owner": "platform"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Resilience Desk",
    "server-id": "prod",
    "properties": {
      "environment": "demo"
    }
  }'

curl -s -X PUT http://127.0.0.1:3000/management/v1/projects/resilience/
warehouses/local \
  -H 'content-type: application/json' \
  -d '{
    "display-name": "Local QGLake Warehouse",
    "storage-root": "file:///tmp/lakecat/qglake",
    "properties": {
      "querygraph": "enabled"
    }
  }'
```

These writes are not Iceberg table metadata. They are catalog control-plane state. LakeCat persists them durably, records authorization receipts, and writes outbox events. When the outbox drains, server, project, warehouse, and storage-profile changes become catalog graph events; the same management changes also become OpenLineage receipts. QueryGraph can later learn the management shape without requiring every Iceberg client to understand it. Server endpoint URLs are operator-visible management metadata, so LakeCat keeps them deliberately plain: they must be absolute `http` or `https` URLs, and they cannot include query strings, fragments, or URI userinfo. Rejected submissions return `server-endpoint-url-hash=sha256:...` evidence rather than echoing the submitted endpoint. Warehouse replay does not forward the raw storage root to graph or lineage consumers. The drained payload replaces `storage-root` with `storage-root-hash`, so QueryGraph can bind tenant evidence to a configured root without receiving the local filesystem path or bucket URI.

### 1.19.3 Storage Profiles And Credential Roots

Storage profiles bind a warehouse to physical storage roots and credential issuance policy. A local profile can return scoped local file configuration. A remote profile should usually reference a secret store and require TypeSec to authorize issuance before any resolver sees the secret reference. Warehouse storage roots are validated before memory or Turso persistence: query strings, fragments, URI userinfo, and literal or percent-encoded dot path segments fail with `warehouse-storage-root-hash=sha256:...` evidence rather than echoing the submitted root. LakeCat rejects profiles whose declared provider conflicts with the URI scheme of the location prefix, so a credential root cannot claim to be local while pointing at an S3 prefix. Those provider/

location mismatch errors follow the same redaction rule as replay: they name provider labels and a `storage-profile-prefix-hash=sha256:...`, not the raw storage root. When multiple profiles in the same warehouse could match a table, LakeCat uses the longest matching location prefix. If two profiles tie on that longest prefix, LakeCat fails closed rather than guessing which credential root or metadata-object boundary should apply. The ambiguity error reports the competing profile ids and `location-prefix-hash=sha256:...` evidence, not the raw storage root. The location prefix itself must be plainly addressed: LakeCat rejects literal and percent-encoded dot path segments, query strings, fragments, and URI userinfo before the profile can reach memory or Turso persistence. Traversal-shaped or decorated storage-profile prefixes fail with `storage-profile-prefix-hash=sha256:...` evidence rather than echoing the raw prefix, token-like query value, or embedded userinfo. The management route pins the same operator-facing behavior, so a rejected storage-profile upsert does not leak the submitted decorated prefix. It also rejects unsafe issuance-mode combinations: `local-file-no-secret` is for file storage only, while `short-lived-secret-ref` is for configured remote providers such as S3, GCS, and Azure. Those mismatches fail with the same `storage-profile-prefix-hash=sha256:...` anchor and without echoing the raw storage prefix or submitted `secret-ref`, so operators can correlate the credential-root error without turning the management API into a credential leak. The `public-config` map is only for non-secret routing hints such as region, endpoint labels, and operational purpose. LakeCat rejects secret-looking public keys and values, so raw tokens, passwords, access keys, and credential query parameters must move behind `secret-ref` and the TypeSec-authorized resolver path. That rule is enforced both when a profile is built from a management request and when a storage profile is revalidated before memory or Turso persistence, so deserialized control-plane records cannot bypass the public-config guard. Public-config validation failures also use `public-config-key-hash=sha256:...` evidence rather than echoing the submitted key or value, because even a rejected key name may contain a secret-looking identifier. LakeCat also reserves credential-evidence keys such as `lakecat.storage-profile-id`, `lakecat.storage-provider`, `lakecat.credential-mode`, and `lakecat.max-credential-ttl-seconds`; operators may still publish non-secret hints such as `lakecat.endpoint`, but they cannot shadow catalog-owned proof in the eventual credential response. The `secret-ref` field itself must remain a clean external secret-store locator: LakeCat rejects query strings, URI fragments, and userinfo before persisting a storage profile, so token-like material cannot hide inside a decorated secret URI. It also rejects literal and percent-encoded dot path segments, so a credential root cannot rely on traversal-like spelling before a resolver sees it. Unsupported credential-root schemes and malformed secret-root paths are rejected with `secret-ref-hash=sha256:...` evidence instead of echoing the submitted secret reference. The same hash-only rule applies to invalid secret-ref URI syntax, decorated URI forms, and embedded secret-like material such as password or token assignments. Management upsert and list responses follow the same redaction rule. They do not echo the raw `secret-ref`; they return `secret-ref-present`, `secret-ref-provider`, and `secret-ref-hash` so operators and QueryGraph can verify that a credential root exists and correlate it without learning the secret-store path. When LakeCat selects the storage profile for a table, the location prefix is also matched on a storage-root boundary. A profile for `s3://lakecat/events` applies to that exact root and to children such as `s3://lakecat/events/tenant-a/table`, but it does not apply to a sibling path such as `s3://lakecat/events-shadow/table`. That keeps

credential roots from accidentally governing more storage than their configured prefix describes; if no stored profile matches, LakeCat falls back to an inferred governed-read profile for the table location. The same check runs after the credential issuer returns: `loadCredentials` rejects any returned prefix broader than the selected profile before LakeCat attaches canonical response evidence, so a custom issuer cannot widen catalog-owned storage scope.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/local-
events \
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "file:///tmp/lakecat/qglake/events",
    "provider": "file",
    "issuance-mode": "local-file-no-secret",
    "public-config": {
      "lakecat.purpose": "developer-loop"
    }
  }'

curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/storage-profiles/s3-events
\
  -H 'content-type: application/json' \
  -d '{
    "location-prefix": "s3://lakecat/events",
    "provider": "s3",
    "issuance-mode": "short-lived-secret-ref",
    "secret-ref": "vault://kv/lakecat/events",
    "public-config": {
      "lakecat.region": "us-west-2",
      "lakecat.purpose": "production-events"
    }
  }'
```

The catalog row stores the public profile and secret reference, not raw cloud keys. A later credential request is checked against TypeSec and against the effective read restriction for the target table. Agents with fine-grained table restrictions are steered to governed Sail-planned reads instead of raw credentials. Trusted humans can receive audited standard credentials only when policy allows the exception. For production secret managers, LakeCat keeps a provider-dispatch seam rather than hard-coding credentials into catalog state. `vault://` can resolve through the built-in Vault HTTP backend when Vault environment configuration is present. `aws-sm://`, `gcp-sm://`, and `azure-kv://` can dispatch to explicitly configured provider backends after TypeSec authorizes the exact secret-ref resource. If no backend is configured, those providers fail closed with an operator-readable not-configured error, and denied TypeSec decisions do not call the backend at all. Configured provider backends receive the same policy-derived `max-credential-ttl-seconds` cap that LakeCat records in the read restriction, and returned credentials must preserve that cap in `lakecat.max-credential-ttl-seconds`. LakeCat rewrites duplicate TTL

config entries into one effective value before returning credentials, preserving a stricter issuer TTL when it is valid and otherwise falling back to the policy cap. It also rewrites LakeCat-owned profile, provider, mode, principal, and governed-read-required evidence after issuance, so a cloud secret backend cannot make the response look like a different catalog decision. The service tests for the REST credential endpoint prove this response shape directly, not just through helper functions. The issuer also rejects any credential whose returned prefix is outside the storage profile's `location-prefix`, so a misconfigured cloud secret backend cannot widen a table's storage scope after TypeSec has authorized the secret reference. The audit event for the credential attempt records redacted `credential-response-evidence`: the response prefix is hashed, LakeCat-owned proof fields are kept as canonical values, and issuer-owned config is hashed rather than copied. That keeps OpenLineage and QueryGraph replay useful without turning lineage into a credential leak. The storage-profile and credential-vend service tests pin that producer-side `location-prefix-hash` evidence is already a full SHA-256 digest before QGLake receives the compact `locationPrefixHash` proof. The trusted-human credential-vending route test pins that the committed outbox payload contains this redacted proof for the audited raw-credential exception path. The blocked-agent route pins the other side of the same contract: when Sail-planned reads are required and no raw credentials are returned, the outbox records an explicit empty `credential-response-evidence` array rather than leaving replay to infer why no credential proof exists. A not-configured resolver error reports the provider label and a `secret-ref-hash=sha256:...` value, not the raw secret URI, so the operator can correlate configuration without leaking the credential root. Resolver validation errors for malformed Vault and TypeSec environment references follow the same rule: wrong schemes, missing Vault mounts or paths, and invalid environment-variable names produce hash evidence instead of echoing the malformed secret reference. Generic provider detection and resolver URI parsing follow that rule too, including unsupported provider schemes, so malformed credential-root strings cannot leak through production resolver diagnostics. Once a configured resolver is authorized to run, backend lookup and secret payload parse failures still stay hash-only: LakeCat returns the secret-reference hash and an error-detail hash instead of the environment variable name, Vault path, token, namespace, backend exception text, or malformed secret fields. When storage-profile changes replay into lineage/OpenLineage evidence, LakeCat does not forward the full secret-store URI. The replay payload keeps `secret-ref-present` and `secret-ref-provider` so QueryGraph can verify that a production credential root exists without learning the Vault, cloud secret manager, or TypeSec environment path. It also replaces the raw storage `location-prefix` with `location-prefix-hash` before graph and lineage projection, so replayed evidence can bind a credential root to a storage scope without exposing the bucket, path, or local filesystem root to downstream consumers. Warehouse replay follows the same shape: `storage-root` becomes `storage-root-hash` before graph and lineage projection, keeping the tenant root replayable without exposing the raw root itself. The drain summary lifts the same proof into compact fields alongside the profile id and provider. QGLake replay verification requires that compact storage-profile upsert evidence, which means a saved handoff can prove the credential root was configured without handing the next system a secret path.

### 1.19.4 A PySpark User Reads Iceberg

A PySpark user should not need to know about QueryGraph. They configure Spark's Iceberg REST catalog and point it at LakeCat:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("lakecat-events")
    .config("spark.sql.catalog.lakecat", "org.apache.iceberg.spark.SparkCatalog")
    .config("spark.sql.catalog.lakecat.type", "rest")
    .config("spark.sql.catalog.lakecat.uri", "http://127.0.0.1:3000/catalog/v1")
    .config("spark.sql.defaultCatalog", "lakecat")
    .getOrCreate()
)

events = spark.table("default.events")
events.select("event_id", "severity").where("severity = 'critical'").show()
```

For an unrestricted principal, the flow looks like a normal Iceberg read:

1. Spark asks LakeCat to resolve `default.events`.
2. LakeCat checks the principal and table capability.
3. LakeCat returns an Iceberg-compatible table response.
4. Spark plans the read using Iceberg metadata.

For a governed principal, the more interesting path is server-side planning. The user request still looks ordinary, but LakeCat derives the mandatory restriction before Sail sees the plan. If policy allows only `event_id` and `severity`, then a wider client projection is narrowed:

```
client asks:     event_id, severity, raw_payload
policy allows:   event_id, severity
Sail receives:   event_id, severity
```

The catalog does not trust the client to remember that. The restriction is re-applied when scan tasks are fetched, and the audit payload records the policy hash, narrowed columns, row predicate, storage location, metadata location, principal, requested stats fields, and effective stats fields. The fetch response also carries LakeCat extension evidence for the exact `required-projection` and `required-filters` derived from the authorized capability. That makes a stateless `fetchScanTasks` replay prove the restriction was re-applied, not merely that the original policy object was echoed. The QGLake fixture verifier checks those fields directly when it fetches scan tasks, so a local acceptance run fails if the response drops either the narrowed projection or the mandatory row predicate proof. The same verifier now checks that the exported policy binding, scan planning extension, and fetch extension all preserve the server-derived purpose and policy-derived `max-credential-ttl-seconds` cap before compact replay proof can be accepted. The scan-planned replay proof also carries `plannedRequestedStatsFields` and `plannedEffectiveStatsFields`, so QGLake can prove a broader stats request, the server-derived narrowing, and the final effective stats fields that match the allowed columns. Saved handoff

summaries and captured replay output are rejected if those fields disappear, widen, or drift apart. It also cross-checks the embedded ODRL policy projection against the structured binding so QueryGraph cannot import a stale copy while LakeCat verifies a different one.

### 1.19.5 A Notebook Requests Credentials

Credential vending is deliberately different from scan planning. Returning storage credentials gives the client broader power than returning a governed task list, so LakeCat treats it as an exception path.

```
curl -s \
  -H 'x-lakecat-principal: agent:triage' \
  http://127.0.0.1:3000/catalog/v1/local/namespaces/default/tables/events/
credentials
```

For an agent bound by a fine-grained restriction, LakeCat should fail closed:

```
{
  "credentials": [],
  "lakecat:credential-block-reason": "fine-grained read restriction requires
Sail-planned reads"
}
```

That empty credential response is not a missing feature. It is the intended agentic posture. The agent should ask LakeCat to plan the read through Sail, not receive raw storage reach. The audit event records the decision and the lineage outbox can replay the credential-vend attempt with the same block reason.

For a trusted human principal, policy can allow an audited raw credential exception. LakeCat still records the same context:

```
principal: analyst:maya
principal-kind: human
table: local.default.events
decision: raw credential exception allowed
reason: trusted human principal
restriction: present in receipt
lineage: credential vend attempted
```

The contrast matters for operators. They can prove that agents were kept on the governed path while a human exception was explicit, policy-backed, and replayable.

### 1.19.6 A View Becomes Part Of The Catalog Story

Views are catalog objects too. LakeCat stores durable view records with SQL, dialect, schema version, typed columns, properties, creator, and warehouse scope. They can be managed through the management API or through warehouse-prefixed catalog routes.

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
```

```
  -d '{
    "sql": "select event_id, severity from default.events where severity =
'\''critical'\''",
    "dialect": "spark-sql",
    "schema-version": 1,
    "columns": [
      {
        "name": "event_id",
        "data-type": {"type": "long"},
        "nullable": false,
        "comment": "Stable event identifier"
      },
      {
        "name": "severity",
        "data-type": {"type": "string"},
        "nullable": false,
        "comment": "Operational severity"
      }
    ],
    "properties": {
      "owner": "resilience-desk"
    }
  }'
```

This is not Iceberg business metadata glued onto a table. It is catalog state about a view object. LakeCat records `view.listed`, `view.upserted`, `view.loaded`, and `view.dropped` audit events. Outbox replay projects listing reads to namespace-scoped graph and OpenLineage evidence, and projects single-view changes and reads to catalog-facing View graph events plus LakeCat OpenLineage view dataset receipts. QueryGraph bootstrap can then include views with OSI hashes, store-assigned view versions, view-aware graph edges, and OpenLineage view counts. The lineage-drain summary also carries compact view replay identity:

```
{
  "name": "events_view",
  "view-version": 1,
  "schema-version": 1,
  "dialect": "sql"
}
```

That `view-version` is assigned by the durable store on each upsert, not by the caller. It is the first compatibility bridge toward Iceberg view commit history: QueryGraph can compare a bootstrap view artifact with the catalog's current view version today. A caller that wants optimistic commit behavior can include `expected-view-version` on the next upsert:

```
curl -s -X PUT \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view \
  -H 'content-type: application/json' \
  -d '{
```

```
    "sql": "select event_id from default.events where severity =
'\''critical'\''",
    "dialect": "spark-sql",
    "schema-version": 2,
    "expected-view-version": 1
  }'
```

If another writer has already advanced the view, LakeCat returns a conflict before it replaces the current view or appends a receipt. Omitting the field keeps the compatibility behavior: the store assigns the next version. The guard must be a positive store-assigned version. `expected-view-version=0` is rejected as a bad request before LakeCat changes the active view or appends any view-version receipt. The same guard can protect deletion:

```
curl -s -X DELETE \
  'http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view?expected-view-version=2'
```

If the current view is no longer version 2, LakeCat returns a conflict before it removes the view or appends a tombstone receipt. Accepted guarded mutations also carry their `expected-view-version` into the audit/outbox payload. During lineage drain, LakeCat turns that into compact replay evidence, so QueryGraph can distinguish "the replacement happened at version 2" from "the replacement was guarded by version 1 and then produced version 2."

LakeCat also writes a compact view-version receipt in the durable store. The receipt records the stable view id, assigned version, previous version, previous receipt hash, content hash, principal, operation, and timestamp. That makes the compact receipt list a hash chain: version 2 points at the version 1 receipt hash, and a later tombstone points at the last upsert receipt hash. Fuller version-log semantics remain a Sail-aligned implementation target. When a view is dropped, LakeCat appends a compact tombstone receipt instead of inventing a new view version: the receipt keeps `view-version` at the last durable version, sets `operation` to `drop`, links to the previous receipt, and preserves the last content hash so QueryGraph or an operator can prove which catalog view state was removed.

```
{
  "event-type": "view.upserted",
  "view-warehouse": "local",
  "view-namespace": ["default"],
  "view-name": "events_view",
  "view-stable-id": "lakecat:view:local:default:events_view",
  "view-version": 2,
  "expected-view-version": 1,
  "graph-events": 2,
  "lineage-events": 1,
  "replay-event-hashes": ["sha256:..."],
  "replay-open-lineage-hashes": ["sha256:..."]
}
```

A `view.listed` replay is intentionally namespace-scoped. It records the warehouse, namespace, view count, graph and lineage projection counts, and receipt hashes for the list read without fabricating a single `view-stable-id`.

QGLake acceptance compares both that `view-stable-id` and `view-version` with the accepted QueryGraph bootstrap view artifacts. That closes a small but important gap: the bootstrap bundle may say a view was exported, and the drain evidence can now prove the corresponding view catalog event was replayed at the same durable catalog version with graph and lineage receipts. When the event was guarded, QGLake replay JSON also includes `expectedViewVersion`, preserving the optimistic version that LakeCat checked before accepting the mutation. The live QGLake fixture uses that path for deletion: after QueryGraph accepts the transient view in the bootstrap bundle, the fixture drops it with `expected-view-version` equal to the accepted durable view version.

When QueryGraph bootstrap is replayed through the outbox, LakeCat includes only compact receipt hashes:

```
{
  "event-type": "querygraph.bootstrap",
  "view-artifact-count": 1,
  "view-version-receipt-hashes": ["sha256:..."]
}
```

QGLake acceptance requires one non-empty receipt hash for each accepted view artifact. That keeps normal Iceberg view access standard, but gives the QueryGraph handoff a durable proof that the exported view version came from LakeCat's catalog spine. Compact handoff verification also binds those bootstrap receipt hashes to `viewReceiptChainProof.views[].acceptedReceiptHash` exactly, so a saved summary cannot splice a valid-looking QueryGraph bootstrap receipt array from another accepted view proof. The fixture also exercises the deletion side of the same workflow: it creates a transient view, accepts a QueryGraph bootstrap that contains that view, drops the view, reads the receipt chain through the governed management endpoints by view name and namespace, and then requires lineage-drain replay to include `view.dropped`, `view.version-receipts-listed`, and `view.version-receipt-chains-listed` evidence with non-empty tombstone receipt hashes and namespace chain hashes. LakeCat also validates the ordered `previous-receipt-hash` links before marking a namespace chain as `chain-verified`, so QueryGraph can reject a replay that contains hashes but not a coherent chain.

The verifier is fail-closed on version progression too. The first receipt must be a version-1 upsert with no previous version or receipt hash; zero-version chains, first-receipt tombstones, and first receipts with forged previous-link fields fail before the chain can be marked verified. Later upserts must point at the previous receipt hash and advance exactly one durable view version. Drop tombstones must point at the previous receipt hash, cite the previous view version, and keep `view-version` equal to the accepted version that was deleted. Unsupported operations and forged `previous-receipt-hash` links fail the same check. That lets QueryGraph reject a chain that is cryptographically linked but lies about how the catalog view advanced.

QueryGraph and operators can also read the compact receipt chain directly from the governed management surface:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/views/
events_view/version-receipts \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```

```
{
  "receipts": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": null,
      "operation": "upsert",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    },
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "view-version": 1,
      "previous-view-version": 1,
      "previous-receipt-hash": "sha256:...",
      "operation": "drop",
      "view-hash": "sha256:...",
      "receipt-hash": "sha256:..."
    }
  ]
}
```

The response is catalog evidence, not Iceberg table metadata. It lets QueryGraph verify the version chain, including tombstones after the current view row is gone, while keeping the richer view history model available for a future Sail-owned implementation.

When the caller needs discovery rather than a known view name, the management surface can return all view receipt chains in a namespace, including chains for views that no longer appear in the active view list:

```
curl -s \
  http://127.0.0.1:3000/management/v1/warehouses/local/namespaces/default/view-
version-receipt-chains \
  -H 'x-lakecat-agent-did: did:example:resilience-agent'
```

```
{
  "chains": [
    {
      "stable-id": "lakecat:view:local:default:events_view",
      "chain-hash": "sha256:...",
      "chain-verified": true,
      "latest-view-version": 1,
      "latest-operation": "drop",
```

```
      "tombstoned": true,
      "receipt-count": 2,
      "receipts": ["..."]
    }
  ]
}
```

That tombstone read is replayable too. LakeCat projects `view.version-receipts-listed` and `view.version-receipt-chains-listed` as lineage evidence, not as graph topology. The graph taxonomy stays in Grust; LakeCat only proves that the governed read saw the tombstone receipt needed to explain why a previously accepted view is now deleted. The namespace response also carries a deterministic `chain-hash` over the chain identity and ordered receipt hashes, and lineage-drain summaries replay that value as `view-version-receipt-chain-hashes` together with a verified-chain count. The QGLake fixture now fails if the namespace-level receipt-chain read, its chain hash, or its verified-chain evidence is absent from lineage-drain replay, so QueryGraph acceptance can depend on compact chain evidence without scraping store internals.

### 1.19.7 QueryGraph Bootstrap

QueryGraph should import LakeCat facts through a verified handoff, not by scraping service internals. The bootstrap endpoint publishes a bundle with artifact hashes:

The exported graph includes a tenant spine:

```
Catalog HAS_SERVER Server
Server HAS_PROJECT Project
Project HAS_WAREHOUSE Warehouse
Warehouse HAS_NAMESPACE Namespace
```

When LakeCat has durable management rows, those graph nodes come from the stored `ServerRecord`, `ProjectRecord`, and `WarehouseRecord`. That means a QueryGraph import can see the real server id, project id, warehouse id, display names, and tenant relationships that operators manage through LakeCat's management API. Replay evidence deliberately redacts storage roots and server endpoint URLs into hashes, so QueryGraph can prove which management state was observed without inheriting local paths, bucket roots, query tokens, URI fragments, or userinfo. In the bootstrap graph, `Server` nodes carry `endpointUrlHash` and `Warehouse` nodes carry `storageRootHash`; they do not carry the raw endpoint URL or storage root. The projection code and service route tests both pin those emitted fields as full SHA-256 digest evidence, so the producer and verifier agree on the shape before QueryGraph import. Authorized management responses can still show the configured endpoint URL or storage root to an operator; graph and lineage replay receive hash-only evidence. When those records do not exist yet, LakeCat falls back to the old deterministic default anchors so bootstrap remains compatible with minimal embedded tests and older import flows.

LakeCat also keeps the older `Catalog HAS_NAMESPACE Namespace` edge in the bundle so existing QueryGraph importers can keep working while newer flows read the tenant path. The tenant anchors and warehouse-to-namespace edges are part of the manifest-covered graph hash, so an importer or handoff verifier can reject a bundle whose namespace is silently detached from the

warehouse or rebound to a different durable tenant chain. The local QGLake verifier enforces that shape: an accepted bootstrap must prove `Catalog -> Server -> Project -> Warehouse -> Namespace -> Table`, not merely that a table node exists somewhere in the graph. It also rejects bundles whose tenant graph nodes expose raw `endpointUrl` or `storageRoot` properties, even if the graph hash and bundle hash have been recomputed around those raw values. Accepted handoffs must use hash-only `endpointUrlHash` and `storageRootHash` evidence for those roots. Those fields are not merely labels with a `sha256:` prefix: the QGLake verifier requires a full 64-hex SHA-256 digest, so an importer cannot accept a rewritten bundle that replaces raw roots with placeholder hash text after the fact. The saved handoff verifier repeats that check against the archived `lakecat-bootstrap.json` artifact, so a later replay cannot pass with a compact summary whose bundle file has lost the tenant path or drifted from the summary hashes.

```
curl -s \
  -H 'x-lakecat-principal: agent:querygraph-importer' \
  http://127.0.0.1:3000/querygraph/v1/bootstrap \
  -o target/qglake/lakecat-bootstrap.json
```

The bundle contains catalog tables, views, policy bindings, graph artifacts, OpenLineage artifacts, Croissant/CDIF/OSI/ODRL projections, and a manifest that hashes what was emitted. The manifest is the import contract. QueryGraph can refuse a bundle whose graph hash, OpenLineage hash, table artifact hash, view artifact hash, or QueryGraph import-compatibility hash does not match. For view-bearing bundles, the import contract also carries compact receipt evidence for each exported view version:

```
{
  "querygraph-import": {
    "schema-version": "lakecat.querygraph.import-compat.v1",
    "view-count": 1,
    "view-receipt-evidence": [
      {
        "stable-id": "lakecat:view:local:default:events_view",
        "view-version": 1,
        "receipt-hash": "sha256:...",
        "receipt-chain-hash": "sha256:..."
      }
    ],
    "view-receipt-evidence-hash": "sha256:..."
  }
}
```

That gives QueryGraph a manifest-covered way to reject a view bootstrap that lost the accepted catalog receipt or detached the ordered receipt chain before the richer graph import begins.

The QueryGraph side should verify the same bundle before importing it:

```
cd /Users/alexy/src/querygraph/qg-rust

cargo run -- lakecat-verify \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json
```

```
cargo run -- lakecat-import \
  --bundle /Users/alexy/src/lakecat/target/qglake/lakecat-bootstrap.json \
  --output .querygraph/lakecat/import-plan.json
```

The importer checks the outer bundle hash, the manifest hashes, the QueryGraph-import compatibility hash, the graph hash, and view receipt evidence, including the accepted receipt hash and receipt-chain hash for each exported view. The graph envelope must be valid as a graph, not just valid JSON: for example, a table and a view in the same namespace must share one namespace node, not emit duplicate vertex ids. That validation belongs on the QueryGraph/Grust side, while LakeCat is responsible for producing a clean catalog-facing graph projection.

For the full local handoff, LakeCat carries a script that runs both sides without writing generated artifacts into the QueryGraph checkout:

`scripts/qglake-handoff-local.sh`

The script starts LakeCat on `127.0.0.1:18181`, uses a Turso-backed local store under `target/qglake-handoff/`, generates the paired QGLake bootstrap bundle and lineage-drain response, runs `qglake-verify-replay`, then runs QueryGraph's `lakecat-verify` and `lakecat-import` against the same bundle. The resulting import plan is written to `target/qglake-handoff/querygraph-import-plan.json`. The same run also writes `target/qglake-handoff/handoff-summary.json`, a compact machine-readable record under the `lakecat.qglake.handoff-summary.v1` schema. It records the catalog URL, principal, table scope, LakeCat replay status from `lakecat.qglake.replay-verification.v1`, QueryGraph-verified table/view counts, and semantic bundle/graph/OpenLineage/import hashes plus standards accepted only after LakeCat replay, `lakecat-verify`, and `lakecat-import` agree. Before writing that compact summary, the harness also requires the governed scan replay evidence to include planned requested/effective projection, planned requested/effective stats fields, fetched required/effective projection, and fetched filters, with effective projection matching the server-derived read restriction. The compact verifier requires the catalog URL to be an absolute HTTP(S) endpoint and requires warehouse, namespace, and table scope to be present before accepting the summary. It rejects captured QueryGraph verify/import output whose warehouse no longer matches the summary. It also embeds `querygraphVerification.verifiedTables` and `verifiedViews` directly in the compact summary. `verifiedTables` must include the stable LakeCat table id derived from that scope, such as `lakecat:table:local:default:events`; `verifiedViews` must include every accepted stable view id from LakeCat replay, such as `lakecat:view:local:default:active_customers_view`; and both arrays must match the QueryGraph table/view counts. Captured QueryGraph verify/import output must match those compact arrays exactly, which keeps a verified artifact set from being replayed against the wrong catalog tenant, table, or view. The `querygraphImportVerification` object is also a compact proof, not only a boolean: it repeats the QueryGraph import table/view ids, counts, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards, and LakeCat rejects a summary unless those fields are SHA-256-shaped and match both `querygraphVerification` and the captured `lakecat-import` output. It also records structured request-identity, scan, management, credential, table-commit,

and view replay evidence, plus compact `requestIdentityProof`, `queryGraphBootstrapProof`, `governedScanProof`, `tableCommitHistoryProof`, `viewReceiptChainProof`, `managementProof`, `storageProfileUpsertProof`, and `credentialVendingProof` objects that lift the replay principal proof, QueryGraph bootstrap/import proof, governed scan counts, pointer-log read proof, view version and receipt-chain proof, management-list counts, redacted credential-root proof, and credential-vending decision out of the full replay tree. The identity proof shows the principal subject and kind used for the replay, the request-identity source and state, the authorization receipt hash, and sanitized TypeDID envelope/proof hashes when a TypeDID envelope is present. The local QGLake fixture currently records the agent-header source with null TypeDID hash slots; a future TypeDID-envelope run can fill those slots without changing the handoff schema. The QueryGraph bootstrap proof shows the accepted bundle, graph, OpenLineage, and QueryGraph import hashes, the table and view artifact counts, standards, policy-binding count, agent delegation and summary signature hashes, view-version receipt hashes, and replay/OpenLineage sink hashes. Those core bootstrap hash anchors must also be SHA-256-shaped before the verifier compares them with QueryGraph's verify/import proof. The scan proof shows LakeCat planned and fetched scan tasks through the governed path, including file, delete-file, and child plan-task counts with replay and OpenLineage hashes. The commit-history proof shows the catalog pointer log was read back with commit count, sequence numbers, commit hashes, positive graph event evidence, and replay/OpenLineage hashes. The view receipt-chain proof shows QueryGraph's accepted view versions together with accepted receipt hashes, accepted `expectedViewVersion` guard evidence when a mutation was guarded, tombstone receipt hashes, namespace chain hashes, verified-chain counts, positive graph event evidence for accepted view replay, and replay/OpenLineage hashes. The credential proof shows the restricted agent was blocked onto Sail-planned reads while the trusted human path used the audited raw-credential exception. The summary also records artifact paths, raw file hashes, captured LakeCat replay output, captured LakeCat handoff-summary verification output, captured QueryGraph verification output, captured QueryGraph import output, captured-output hashes for the LakeCat replay and QueryGraph verify/import JSON files, and service log path. The handoff verifier does not stop at byte hashes: it parses the saved LakeCat replay JSON and QueryGraph verify/import JSON captures and checks their replay schema/status, table and view counts, warehouse, verified table ids, bundle hash, graph hash, OpenLineage hash, QueryGraph import hash, and standards against the compact summary. It compares those standards across sections and independently requires the full QGLake standards set, so a compact handoff cannot omit ODRL or another required contract simply by making every section omit it consistently. It compares the captured LakeCat `replay-evidence.requestIdentity` and `replay-evidence.queryGraphBootstrap` objects with the compact request-identity and bootstrap proofs, including the principal, authorization hash, TypeDID hash slots, delegation and summary signature hashes, artifact counts, standards, replay hashes, and the accepted bundle, graph, OpenLineage, and QueryGraph import hashes. The compact verifier also requires the bootstrap proof to carry the same request-identity source and verification state as `requestIdentityProof`. The authorization receipt hashes are intentionally distinct proof slots: `requestIdentityProof` records the lineage-drain read receipt, while `queryGraphBootstrapProof` records the original bootstrap event receipt. The verifier requires both hashes to be SHA-256-shaped and bound back to their captured replay sections rather than forcing them to be equal. The compact verifier

also validates the TypeDID hash-slot shape directly: envelope and proof slots must be null or SHA-256 hashes, and a TypeDID proof hash cannot appear without the paired envelope hash. As with authorization receipts, the request and bootstrap TypeDID hash slots are independently shaped replay evidence because they may come from different requests in the captured workflow. That keeps the compact handoff self-describing without moving TypeDID trust semantics out of TypeSec. Live request parsing now enforces the same boundary earlier: a caller that sends `x-lakecat-typedid-proof` without `x-lakecat-typedid-envelope` receives a hash-only `typedid-proof-hash` rejection, and a caller that sends agent delegation or agent summary proof headers without an agent-shaped identity receives only the matching proof hash. Those failures happen before governance or capability receipt creation, so raw proof material cannot become either policy context or operator-facing diagnostics. TypeDID verifier failures follow the same rule: malformed or rejected envelopes report only the envelope hash and error-detail hash, and a verified-subject mismatch reports only hashes of the verified and supplied principals before governance dispatch. LakeCat applies that redaction at the verifier trait boundary, so a custom TypeDID verifier can choose the error class without leaking raw envelope, DID, gateway, or payload text into the catalog response. The JSON output from `lakecat-cli qglake-verify-handoff` also carries the accepted lineage-drain identity source, identity state, and TypeDID envelope/proof hash slots in `lineageDrainArtifactSemantics`, so QueryGraph can index the verified drain boundary without reparsing the raw drain artifact. If a saved `lakecatHandoffVerifyOutput` artifact is present, LakeCat binds those saved drain identity semantics back to the compact `requestIdentityProof`, so a rehash cannot disguise drift in principal, authorization receipt, source/state, or TypeDID hash-slot evidence. It compares captured `replay-evidence.scan` with `governedScanProof`, requiring positive plan task, scan-plan graph event, file task, delete file, and child plan task counts plus the planned and fetched read-restriction objects and the fetch-side required projection/filter evidence. The verifier rejects a summary if the fetched restriction drifts from the planned restriction, so the compact handoff proves the narrowed allowed columns, row predicate, and policy hashes alongside the planned/fetched replay and OpenLineage hashes that prove the Sail-planned read path. The compact Rust verifier requires both the planned and fetched OpenLineage hashes directly, so automation can reject incomplete scan lineage without falling back to the shell harness. It also compares the captured `replay-evidence.tableCommitHistory` object with `tableCommitHistoryProof`, including the commit count, sequence numbers, commit hashes, graph event count, replay hashes, and OpenLineage hashes that prove the pointer-log commit history was not rewritten between replay and summary and that the commit-history replay projected catalog graph evidence. The compact verifier also requires the commit count to match the sequence-number and commit-hash arrays, requires every sequence number to be positive and strictly increasing, and requires positive graph event evidence plus replay and OpenLineage receipt hashes. It compares the captured `replay-evidence.views` object with `viewReceiptChainProof`, including accepted view receipts, accepted-view graph event counts, expected-version guard evidence, tombstone receipts, namespace receipt-chain hashes, and their replay/OpenLineage hashes, so durable view history stays tied to the saved LakeCat replay artifact. It also compares the tombstone branch's `expectedViewVersion` with the accepted view version, so a handoff cannot claim a governed deletion unless the saved LakeCat replay proves that deletion used the catalog's optimistic version guard; the standalone `qglake-verify-`

handoff command enforces this match even when a summary is checked outside the local shell harness. The same standalone verifier now requires the compact view proof to keep viewCount consistent with the accepted view list, carry stable warehouse/namespace/name identity, prove viewVersion == acceptedViewVersion, and carry accepted receipt hashes, accepted receipt-chain hashes, positive accepted-view graph event evidence, tombstone receipt hashes, positive verified-chain counts, receipt-chain warehouse/namespace identity, namespace chain hashes, and replay/OpenLineage hashes. The verifier also checks that each namespace receipt-chain summary's verifiedChainCount equals the number of chain hashes and that the receipt hashes cover those chains. It also requires queryGraphBootstrapProof.viewVersionReceiptHashes to match the accepted view receipt hashes exactly, so the compact summary cannot combine bootstrap view receipt evidence from one run with accepted-view proof from another. For active accepted views, it additionally requires acceptedReceiptChainHash to appear in the namespace chainHashes evidence, so a compact summary cannot pair a valid-looking accepted view receipt with an unrelated namespace receipt-chain proof. For tombstoned accepted views, the accepted chain can be a prefix of the later tombstone chain, but the compact proof must include tombstone receipt evidence whose expectedViewVersion preserves the accepted view version. A consumer can reject a handoff whose view history claim lacks identity, accepted-version, count-aligned hash-chain evidence, active-chain coverage or tombstone guard evidence, or replay evidence before parsing the full replay tree. It compares captured LakeCat replay replay-evidence.management list counts with compact lakecatReplayVerification.managementProof, requiring positive server, project, warehouse, and storage-profile counts, positive graph event counts for server, project, warehouse, policy-binding, and storage-profile list replay, and a policy-binding count that matches the QueryGraph bootstrap proof. Those management-list counts are receipt-backed: the compact proof and captured replay must also agree on replay and OpenLineage hash arrays for server.listed, project.listed, warehouse.listed, policy-binding.listed, and storage-profile.listed. It also compares the captured LakeCat replay replay-evidence.management.storageProfileUpsert object with the compact lakecatReplayVerification.storageProfileUpsertProof, including the profile id, provider, issuance mode, location-prefix hash, secret-reference presence/provider/hash, replay hashes, and OpenLineage hashes. The compact verifier also requires that location-prefix value to be a SHA-256 hash and requires a redacted secret-reference provider and secretRefHash whenever the proof says a secret reference is present. If the proof says no secret reference is present, the provider and hash must both be null. It also compares the captured replay-evidence.credentials restricted-agent and trusted-human branches with the compact credentialVendingProof, so a saved handoff cannot claim that agents were blocked onto Sail-planned reads or that humans used an audited exception unless the captured LakeCat replay proves the same decision. Each credential branch carries the same redacted storage-scope anchor as the storage-profile upsert proof: locationPrefixHash binds the credential-vend attempt to the configured storage root without replaying the raw prefix. That anchor must be a full sha256:-prefixed 64-hex digest in both the storage-profile proof and each credential branch. LakeCat checks the storage-scope hash at lineage-drain replay time before the compact handoff summary is accepted, and the operator-readable credential replay line prints the same hash so captured terminal output cannot look complete while omitting the credential-root boundary. Source replay validates secret-reference

shape on the credential branches themselves: if a credential-root proof says a secret reference is present, it must carry a non-empty provider and SHA-256 `secretRefHash`; if it says no secret reference is present, provider and hash evidence must be absent. Source replay and compact handoff verification both reserve `rawCredentialExceptionReason` for the audited trusted-human path; a restricted agent proof must be blocked with `blockReason` and cannot carry a raw exception reason. The local handoff harness now preserves the lineage-drain artifact before replay verification, so if any of these proof checks fail the operator still has the exact failed drain JSON for diagnosis. The compact handoff verifier also validates that credential proof directly: the restricted branch must name the accepted agent principal, carry the Sail-planned-read block reason, prove zero credentials, carry the policy-derived `maxCredentialTtlSeconds` cap, explicitly set `rawCredentialExceptionAllowed` to false, reject any non-null `rawCredentialExceptionReason`, and include replay/OpenLineage hashes; the trusted-human branch must name a human principal, prove a positive credential count, carry the same policy-derived TTL cap, carry the exact audited raw-credential exception reason, prove `blockReason` is null, and include replay/OpenLineage hashes. The compact verifier has direct negative coverage for the credential-branch secret-reference rules too, so a handoff cannot hide malformed provider or hash evidence behind an otherwise matching storage-profile upsert proof. The local handoff harness now preserves those replay fields while building the compact summary, rather than relying on the nested raw replay blob alone: scan graph events and fetch-side projection/filter requirements, management graph events, storage-profile graph events, credential block/exception fields, and table commit-history graph events all move into the verifier-facing proof sections before `qglake-verify-handoff` runs. That makes the handoff repeatable from the LakeCat repo while keeping QueryGraph responsible for graph validation and import semantics. The handoff script refuses to write the summary unless LakeCat replay JSON contains request-identity evidence for the expected agent principal, an explicit identity source/state, an authorization receipt hash, and explicit TypeDID envelope/proof hash slots that are either null or SHA-256 hashes. A proof hash is valid only when the matching envelope hash is present. It also refuses to write the summary unless LakeCat replay JSON contains redacted `storageProfileUpsert` evidence with replay and OpenLineage hashes, and the accepted summary repeats that evidence as `lakecatReplayVerification.storageProfileUpsertProof`. QueryGraph gets proof that the credential root was configured, including the provider, issuance mode, the configured location-prefix hash, and a redacted hash of the secret reference, without receiving the underlying secret-store URI or full storage prefix in the compact proof. The operator-readable management replay line now prints the same storage-scope hash and redacted secret-reference state, so a captured transcript cannot describe the credential root only by provider while omitting its redacted storage scope or secret-reference boundary. The script also refuses to write a summary unless LakeCat replay proves both sides of credential vending: untrusted agents get no raw credentials, trusted humans receive only the audited standard exception, and both branches preserve the `max-credential-ttl-seconds` restriction as `maxCredentialTtlSeconds` in compact evidence. For reads, the summary similarly refuses to omit proof that scan planning and scan-task fetch both replayed with sink receipt hashes. The compact scan proof must preserve the server-derived read restriction as a full restriction, not only as columns and filters: allowed columns, row predicate, purpose, policy-hash evidence, and `max-credential-ttl-seconds` must be present, and the planned and fetched restrictions must agree. The fetched required filters must

also be exactly the mandatory row predicate evidence, not a prefix with extra unverified filters appended. For catalog state, it refuses to omit proof that the table commit-history read replayed with sequence-number and commit-hash evidence. For views, it refuses to omit proof that accepted view versions line up with their receipt hashes and that the namespace-level tombstone and receipt-chain reads replayed with chain hashes and verified-chain counts.

This gives the semantic layer a responsible starting point. LakeCat says:

```
Here are the governed catalog objects.
Here are the policies that shaped planning.
Here are stable dataset and field anchors.
Here is the graph envelope.
Here is the OpenLineage replay evidence.
Here are the hashes that bind the handoff.
```

QueryGraph then owns the richer semantic work: metrics, dimensions, joins, business names, multi-dataset reasoning, and agent-facing synthesis.

### 1.19.8 Draining The Outbox

LakeCat records side effects as durable outbox events. Draining the outbox is what turns committed catalog facts into graph and lineage receipts:

```
curl -s -X POST \
  -H 'x-lakecat-principal: agent:lineage-drainer' \
  http://127.0.0.1:3000/management/v1/lineage/drain \
  -o target/qglake/lineage-drain.json
```

A useful drain response includes delivered event types, graph projection counts, lineage projection counts, receipt hashes, and the authorization proof for the drain request itself. That last part is easy to overlook. Reading the replay stream is also privileged, so LakeCat records that the drainer was allowed to read lineage evidence. Standard catalog reads replay too: `catalog.config-read` records a warehouse-scoped graph/OpenLineage fact for the Iceberg REST configuration entrypoint; `namespace.listed` records the namespace listing at the warehouse; and `namespace.loaded` records the specific namespace resolved through the standard catalog route. For view events, the response includes the warehouse, namespace, view name, and QueryGraph-compatible stable id, so downstream acceptance can check replay identity without parsing the full audit payload. Table restores replay as table graph evidence plus a restore OpenLineage receipt, so a soft-deleted table returning to service is visible to QueryGraph without forcing LakeCat to invent restore-specific graph taxonomy. Management list reads for policy bindings, projects, servers, storage profiles, and warehouses replay as OpenLineage receipts too. They intentionally do not create list-specific graph nodes in LakeCat; Grust owns the reusable hierarchy and traversal model. The drain response lifts their counts and management scope into compact fields, so QueryGraph can verify the control-plane read evidence without opening the raw lineage payload. It also carries replay and OpenLineage hash arrays for those management-list reads, so a compact handoff cannot prove only that the right number of management records existed while losing the receipt evidence for the reads. The lineage-drain verifier rejects those source replay events when the receipt arrays are empty or not SHA-256-shaped, so the compact `managementProof` starts from verified

replay evidence rather than normalizing malformed hashes later. It also rejects management-list source replay without catalog graph projection evidence, keeping the durable server/project/warehouse, policy, and storage-profile facts visible to QueryGraph through Grust-facing graph events. Compact `managementProof` carries those graph event counts too, and captured replay agreement checks them, so the graph evidence cannot disappear between source replay and handoff verification. The QGLake acceptance workflow now establishes its server/project/warehouse tenant spine, performs governed server, project, warehouse, policy-list, storage-profile-list, scan-planning, scan-task-fetch, and table commit-history reads before bootstrap, and rejects a drain that does not replay matching `server.listed`, `project.listed`, `warehouse.listed`, `policy-binding.listed`, `storage-profile.listed`, `table.scan-planned`, `table.scan-tasks-fetched`, and `table.commits-listed` evidence. Request-identity and bootstrap replay are checked before any compact handoff proof is built: the drain authorization, bootstrap authorization, QueryGraph bundle/import hashes, agent delegation hash, agent summary signature hash, and TypeDID envelope/proof hashes must be SHA-256-shaped, and a TypeDID proof hash cannot appear without its paired envelope hash. For scan replay, the typed drain summary carries scan-plan task counts, scan-plan graph event evidence, fetched file-scan, delete-file, and child-plan task counts, along with planned and fetched OpenLineage receipt hashes. Source replay validation now also requires planned and fetched read restrictions to match before compact proof generation, requires both requested/effective projection and requested/effective stats-field evidence, requires effective projection to be a narrowed subset of the requested projection and to match the planned allowed columns, and requires effective stats fields to be a narrowed subset of the requested stats fields in both source replay and compact handoff proof. It also requires the fetched projection and filter requirements to exactly preserve the fetched allowed columns and row predicate. A fetched response that omits required-filter proof is rejected just like one that widens or changes that proof, and the compact handoff summary applies the same missing-proof check before accepting governed scan evidence. Credential replay applies the same policy-proof discipline to the two credential branches: the restricted-agent denial and trusted-human audited raw-credential exception must both carry a complete read restriction, and those restrictions must match before credential proof can feed the compact handoff summary. For commit-history replay, the typed drain summary carries the commit count, committed sequence numbers, commit hashes, replay hashes, and OpenLineage hashes. The handoff verifier rejects compact scan proofs without those OpenLineage hashes and compact commit-history proofs whose counts, sequences, or hash arrays do not align. Source replay validation applies the same pointer-history discipline before compact proof generation: the table commit count must match the sequence-number and commit-hash arrays, commit sequences must be positive and strictly increasing, and commit hashes must be SHA-256-shaped before pointer-history evidence can enter the compact handoff proof. QueryGraph can therefore verify the governed Sail-planned read and pointer-history inspection without parsing the full lineage payload. The core QueryGraph bundle, graph, OpenLineage, and import anchors must be SHA-256-shaped in compact verify/import/bootstrap proof before a matching summary can pass; matching strings are not enough. The bootstrap, scan, credential, view, receipt-chain, and commit-history receipt arrays must also be SHA-256-shaped before compact proof generation can consume them. The same shape check applies to accepted-view receipt evidence: bootstrap view-version receipt hashes, tombstone receipt hashes, and namespace receipt-chain

hashes must be SHA-256-shaped before accepted-view proof feeds the compact handoff summary. Dropped and active accepted-view source replay also compares the bootstrap view-version receipt hashes with the accepted QueryGraph verification set, so a valid-looking receipt array cannot be spliced from another bootstrap proof. The compact handoff verifier repeats the same binding against `viewReceiptChainProof.views[].acceptedReceiptHash`, catching drift even when an operator validates only the saved summary. Dropped accepted-view source replay also binds the namespace receipt-chain read back to the accepted view warehouse/namespace and rejects verified-chain count or receipt-hash coverage drift before compact handoff proof is generated. The same replay now emits catalog-facing `Commit` graph events for the listed sequences, leaving traversal and query semantics to Grust.

For handoff testing, LakeCat can verify a saved bootstrap bundle and a saved drain response together:

```
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

That offline check replays the same boundary assertions used by the live QGLake fixture: the bundle manifest must verify, the QueryGraph import compatibility contract must match, and the lineage drain must carry matching bootstrap hashes, credential-denial receipts, management-list evidence, scan/fetch task evidence, and table commit-history receipt evidence, plus view receipt evidence when views are present. On success, the command prints the accepted bundle and QueryGraph import hashes, table/view counts, and compact control-plane lines such as:

```
scan replay plan_tasks=1 plan_graph_events=1 planned_ttl=300
planned_purpose=qglake-agent-demo file_tasks=1 delete_files=1 child_plan_tasks=1
fetched_ttl=300 fetched_purpose=qglake-agent-demo
management replay servers=1 projects=1 warehouses=1 policies=1 storage_profiles=1
storage_profile_upserts=1 credential_root=events-local:file:local-file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
credential replay restricted=blocked:sail-planned-read-required
restricted_count=0 restricted_ttl=300 restricted_profile=events-local:file:local-
file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
human=allowed:trusted-human-audited-raw human_count=1 human_ttl=300
human_profile=events-local:file:local-file-no-
secret:location_prefix_hash=sha256:2222222222222222222222222222222222222222222222222222222222222222
table commit history commits=1 sequences=1 hashes=sha256:... graph_events=1
```

Those lines are intentionally small enough for QueryGraph handoff scripts and operator logs, but they still come from the same typed lineage-drain summaries that the verifier requires before accepting replay. The scan line keeps the planned and fetched credential TTL caps visible beside the task counts, while JSON mode carries the full read-restriction evidence tree. Scan planning records both requested and effective projection evidence; scan-task fetch records the server-derived required projection and mirrors it as `effective-projection`, so replay can compare both stages with the same policy-narrowed vocabulary. QGLake acceptance now rejects handoffs

where the fetched effective projection is missing or drifts away from the fetched read restriction, which means a compact replay summary cannot quietly widen what the server actually planned. The live handoff harness performs that projection check before writing `handoff-summary.json`, so the artifact is born with the same proof shape the verifier later enforces.

After the full local handoff writes `handoff-summary.json`, LakeCat can also verify the compact summary itself:

```
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
```

That command validates the `lakecat.qglake.handoff-summary.v1` schema, QueryGraph verify/import agreement, LakeCat replay agreement, and the compact proof objects for request identity, QueryGraph bootstrap, governed scan, pointer history, view receipt chains, storage-profile upsert, and credential vending. It also recomputes the raw file hashes for the bundle, lineage-drain response, and QueryGraph import plan named in the summary, rejecting stale or tampered artifact files before automation consumes them. It parses the saved bootstrap bundle and reruns the tenant graph and semantic hash verifier. It also parses the saved QueryGraph import plan and requires its embedded verification, table/view stable ids, semantic hashes, standards, and graph node/edge evidence to match the compact QueryGraph import proof. The verifier also compares those QueryGraph import-plan graph node and edge counts with the verified bootstrap bundle graph counts, so an import plan cannot keep the semantic hashes and table/view ids while silently dropping graph material. It also parses the saved lineage-drain response, reruns the typed QGLake replay verifier, and regenerates the LakeCat replay evidence that proves request identity, QueryGraph bootstrap replay, governed scan replay, pointer history, view receipt chains, storage-profile replay, and credential-vending decisions. It then compares that regenerated replay evidence to the compact `lakecatReplayVerification` proof. The governed scan proof includes both the requested and effective stats-field arrays from the scan-planned replay, and the verifier rejects handoffs where the effective fields no longer match the allowed columns or no longer prove policy narrowing. Credential-vending proof is not just a count: each restricted-agent and trusted-human branch carries the redacted `storageProfile` graph anchor and `maxCredentialTtlSeconds`, including profile id, provider, issuance mode, secret-reference presence, and the graph event count emitted by replay. The storage-profile upsert proof also carries its own positive `graphEvents` count, and captured LakeCat replay must match it. The verifier also rejects a handoff when the credential branches do not bind back to the same storage-profile upsert proof: profile id, provider, issuance mode, location-prefix hash, and secret-reference state must all match the replayed management event. A saved handoff is rejected if the archived lineage drain proves lineage receipt hashes but omits that credential TTL cap or credential-root graph projection. It also recomputes the captured LakeCat replay and QueryGraph verify/import output hashes, so terminal captures cannot drift from the compact summary. It compares the legacy string path aliases for the LakeCat replay, QueryGraph verify, and QueryGraph import captures with the hashed `capturedOutputs` paths they duplicate. It also hashes the service log through `serviceLogHash`, so archived operational logs cannot drift behind a stable path. The final local summary also binds the first LakeCat handoff-verifier capture with `lakecatHandoffVerifyOutputHash`. Because that output can only

exist after a successful verifier run, the harness performs a second sidecar self-check: first it writes `target/qglake-handoff/lakecat-handoff-verify.json`, then it records the file's hash in the summary, then it verifies the summary again without overwriting the declared artifact. The verifier checks that saved JSON is a `lakecat.qglake.handoff-verification.v1` success for the same principal, catalog URL, warehouse, namespace, and table, and that its table/view counts, stable ids, standards, request-identity proof, and QueryGraph bootstrap proof still match the compact handoff summary. It also checks the saved self-verifier output's bundle, lineage-drain, QueryGraph import-plan, captured-output, and service-log hashes against the summary's artifact manifest. It also checks the saved self-verifier output's own semantic sections: captured replay semantics must match the compact LakeCat and QueryGraph proof, bundle artifact semantics must match QueryGraph verification, import-plan semantics must match QueryGraph import verification, lineage-drain semantics must match the accepted replay proof, and saved import-plan graph counts must still match the saved bundle graph counts. Then it parses those captured JSON files and checks that the replay schema/status, table/view counts, semantic hashes, standards, request-identity proof, QueryGraph bootstrap proof, governed scan proof, storage-profile upsert proof, and credential-vending proof inside the captures still match the summary. It also rejects malformed TypeDID hash slots in the request-identity and QueryGraph bootstrap proofs before a consumer has to interpret those slots. The local handoff harness runs it automatically and writes the captured verifier output to `target/qglake-handoff/lakecat-handoff-verify.json`.

The end-to-end result is a chain:

```
catalog write
  -> audit event
  -> outbox event
  -> graph projection
  -> OpenLineage projection
  -> QueryGraph import evidence
```

If graph or lineage sinks are down, catalog state should not be lost or rolled back accidentally. The outbox lets LakeCat retry projection from committed state. A drain acknowledges delivery only after every projection in the batch succeeds. If OpenLineage fails after a graph event has already been emitted, the drain fails and the catalog event remains pending, so recovery starts from the committed outbox rather than from guesswork. If the graph sink fails first, LakeCat fails the drain before emitting lineage and still leaves the outbox event pending, so graph and lineage consumers recover from the same committed catalog fact instead of diverging.

Replay order is part of that contract. LakeCat selects undelivered outbox events by `created_at,event_id` in both embedded memory tests and the durable Turso store, so a QGLake replay does not depend on writer interleaving or database row-return quirks. Delivery acknowledgement is duplicate-safe as well: if a drainer accidentally reports the same event id twice, LakeCat marks the event once and the receipt count remains tied to committed catalog facts. Pending batch validation happens before projection: if a store returns duplicate event IDs in the same drain batch, LakeCat fails the drain with only the duplicate event-id hash and does not emit graph, OpenLineage, or acknowledgement side effects for that batch. The same redaction rule applies to malformed pending records. If a custom or corrupted store hands the drain an event

whose payload cannot be projected, LakeCat reports only the outbox event-id hash and stops before graph emission, lineage emission, or delivery acknowledgement. Malformed table and principal identity JSON decode failures follow that same pattern: they carry event-hash evidence for correlation without echoing the raw event identifier into diagnostics.

### 1.19.9 An Agentic QGLake Flow

The agentic path is the reason LakeCat has to be more than a passive catalog. Imagine a resilience supervisor agent investigating incidents:

1. The supervisor delegates table triage to a specialist agent.
2. The specialist asks LakeCat to plan a scan over `local.default.events`.
3. LakeCat resolves the agent identity and TypeDID context.
4. TypeSec authorizes the table scan and returns a restricted capability.
5. LakeCat narrows the projection and appends the required row predicate.
6. Sail plans against the current Iceberg metadata and delete manifests.
7. LakeCat returns governed plan and fetch-task responses.
8. The specialist summarizes only the allowed result shape.
9. LakeCat records scan and credential decisions into audit/outbox.
10. QueryGraph imports graph, policy, lineage, and bootstrap evidence.

The key point is the absence of raw storage reach. The specialist agent does not need broad cloud credentials to do its job. It needs a governed plan, a bounded task set, and a receipt trail.

The local fixture compresses this story into a short artifact-producing sequence:

```
cargo run -p lakecat-cli -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
```

The one-command handoff wraps the same evidence in a live local service run and then asks QueryGraph to verify and import it:

```
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cat target/qglake-handoff/handoff-summary.json
```

That fixture creates the sample table shape, installs a restricted policy, verifies governed scan planning, verifies fetch-scan-task reapplication, checks requested/effective stats-field narrowing in replay and handoff proof, exercises delete manifest handling, probes credential-vend behavior for agents and trusted humans, verifies compact table commit-history evidence, exports QueryGraph bootstrap artifacts, drains the outbox, and proves the resulting bundle through QueryGraph's Rust verifier/importer. It then asks LakeCat to verify its own compact handoff summary and recompute

the raw artifact file hashes, making the summary a first-class acceptance artifact rather than an unchecked convenience file. It is small, but it is not decorative. It is the acceptance story for a catalog that participates in the user workflow from notebook to agent. The summary file gives automation a single stable place to find the accepted table/view counts, semantic hashes, bundle, lineage drain, import plan, captured verifier outputs, and raw artifact hashes without scraping terminal text.

## 1.20 Operating The Book's Example System

The local development posture is intentionally small:

```
cargo run -p lakecat-cli -- config
cargo run -p lakecat-cli -- storage-profile-list
cargo run -p lakecat-cli -- policy-list
cargo run -p lakecat-cli -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json \
  --drain-output target/qglake/lineage-drain.json \
  --principal did:example:agent
cargo run -p lakecat-cli -- qglake-verify-replay \
  --bundle target/qglake/lakecat-bootstrap.json \
  --drain target/qglake/lineage-drain.json \
  --principal did:example:agent
scripts/qglake-handoff-local.sh
cargo run -p lakecat-cli -- qglake-verify-handoff \
  --summary target/qglake-handoff/handoff-summary.json \
  --json
cargo run -p lakecat-cli -- bootstrap-export \
  --output lakecat-bootstrap.json
```

The important thing is what these commands exercise. They are not a separate product surface. They touch the same catalog, policy, bootstrap, and QueryGraph export contracts that the service uses.

## 1.21 What Comes Next

LakeCat is a direction more than a single release. The next slices should keep making the architecture more true:

1. Remove temporary Sail patch bridges when the required helpers are available upstream.
2. Keep pushing reusable Iceberg table-status, planning, metadata-as-data, and commit helpers into Sail.
3. Make the transactional outbox the only path for graph and lineage side effects.
4. Expand TypeSec-backed capability checks until every privileged path is unbypassable.
5. Keep graph taxonomy and Cypher behavior in Grust.
6. Keep OSI as a QueryGraph-owned semantic handoff rather than a LakeCat-authored business model.
7. Prove the bootstrap bundle through QueryGraph import on every meaningful public-surface change.

The payoff is a catalog foundation that feels ordinary to Iceberg clients and rich to QueryGraph. Tables remain portable. Policies become enforceable. Graph and lineage become replayable. Sail plans close to the data. QueryGraph gets a trustworthy substrate for the next version of its lakehouse intelligence.